ASMICORE

AGTP Batch 01

RTL Implementation and Verification: Sequence Detector, Traffic Light Controller & Synchronous FIFO

RTL and DV Team

Submitted by: Mohammed Asfaqul Alam Chy

Program : RTL and DV Team

Submission date: September 1, 2026

Report Details

DV and RTL Team project 03

EXPERIMENT INDEX

Contents

Day 1: Foundations of RTL & Synthesis **3**

Day 2: State Machine Design & Analysis **7**

Day 3: The Verification Mindset & Testbench Architecture **11**

Day 4: RTL Implementation & Initial Simulation **16**

Day 5: Traffic Light Controller — State Machine with Timing **20**

Day 6: Synchronous FIFO — Memory Buffer Implementation **25**

Day 7: Specification Analysis & Corner-Case Identification **31**

Day 8: Specification Documentation & Design Cleanup **36**

Day 9: Feature Refinement, Requirements & Traceability **40**

Day 10: Checker Types, Stimulus & Priority **43**

Day 11: Coverage Theory & Coverage Planning **46**

Day 12: Knowledge-Base Completion & Self-Review **49**

Day 13: Final Team Reflection **51**

Day 14: Presentation Preparation & Final Wrap-Up **53**

Day 1: Foundations of RTL & Synthesis

Objective

State the objective of the experiment here.

Procedure

RTL-to-Transistor Journey: A Synthesis Pipeline

Register Transfer Level (RTL) code, written in a hardware description language such as SystemVerilog, is a behavioral model of digital logic. It is transformed into a physical circuit through a sequence of synthesis and implementation stages.

The synthesis tool first reads and *elaborates* the RTL. During elaboration, it resolves the design hierarchy and parameters and infers registers, multiplexers, and combinational logic. The result is a technology-independent netlist of generic elements such as AND gates, OR gates, and flip-flops. During *logic optimization*, Boolean simplification, redundancy removal, and resource sharing are applied to reduce area and power while satisfying the supplied timing constraints.

In *technology mapping*, the optimized generic netlist is mapped to the standard cells provided by the target technology library. Each selected cell has characterized area, delay, and power properties. The mapped netlist then enters physical design, where place-and-route tools position the cells and connect them using metal layers. The completed layout can subsequently be verified and prepared for fabrication. Thus, the RTL acts as the functional blueprint from which the implementation tools construct the physical circuit.

Annotated Example 1: 8-bit Counter with Enable

An 8-bit up-counter was written to demonstrate sequential RTL, an asynchronous active-low reset, and a synchronous enable input. The annotated SystemVerilog module is shown below.

```
// Module: counter_8bit
// Description: An 8-bit up-counter with a synchronous enable.
// ----------------------------------------------

module counter_8bit (
    // Interface ports
    input  logic       clk,   // Sequential updates occur on the rising edge.
    input  logic       rst_n, // Active-low asynchronous reset.
    input  logic       en,    // High: increment on the next rising edge.
    output logic [7:0] count  // Current 8-bit counter value.
);

    // always_ff models flip-flop-based sequential logic. This block responds
```

```
    // to a rising clock edge and immediately to a falling reset edge.
    always_ff @(posedge clk or negedge rst_n) begin
        // The asynchronous reset has the highest priority.
        if (!rst_n) begin
            count <= 8'h00;
        end
        // With reset inactive, the enable is sampled on the rising edge.
        else begin
            if (en) begin
                // The 8-bit result wraps naturally from 8'hFF to 8'h00.
                count <= $unsigned(count + 1'b1);
            end
            // When en is low, no assignment is made and the flip-flops retain
            // their previous value.
        end
    end

endmodule
```

Annotated Example 2: 2-input AND Gate

A fundamental two-input AND gate was then described using dataflow modeling. This example demonstrates continuous assignment and purely combinational logic.

```
// Module: and_gate
// Description: A 2-input AND gate using a continuous assignment.
// ------------------------------------------------

module and_gate (
    // The inputs drive signals into the module; the output is driven by it.
    input  logic a, // First operand of the AND operation.
    input  logic b, // Second operand of the AND operation.
    output logic y  // Result of the AND operation.
);

    // y is updated whenever a or b changes. Because this is a continuous
    // combinational assignment, it infers an AND gate and no storage.
    assign y = a & b;

endmodule
```

Observations and Results

Behavior of the 8-bit Counter

The counter describes a bank of eight flip-flops with reset and enable control. Driving `rst_n` low clears `count` to hexadecimal `00`, independently of the clock. When reset is inactive and `en` is high, the stored value increases by one at each positive clock edge. When `en` is low, the previous value

is retained. Because the output width is eight bits, incrementing `FF` produces `00`; the carry beyond bit 7 is discarded.

rst_n	Clock event	en	Result for count
0	Any / falling edge	X	Cleared to `8'h00`
1	Positive edge	1	Incremented by one
1	Positive edge	0	Previous value retained
1	No positive edge	X	Previous value retained

Behavior of the 2-input AND Gate

The AND-gate module produced a high output only when both inputs were high. The continuous assignment responds to every input change and does not infer a clock, latch, or flip-flop.

a	b	y = a & b
0	0	0
0	1	0
1	0	0
1	1	1

Understanding `wire` and `logic`

In SystemVerilog, `wire` is a net type that models an interconnection. It is driven by a continuous assignment, a module output, or another structural source and cannot be assigned inside procedural blocks such as `always_ff`. Nets can support multiple drivers when an appropriate resolution model is intended, although unintended multiple drivers normally indicate a design error.

The `logic` type is a four-state variable that can represent `0`, `1`, `X`, and `Z`. It can be used for most ports and internal signals and may be driven either by one procedural block or by one continuous/structural source. A signal declared as `logic` must not have conflicting multiple drivers.

Consequently, `logic` is the preferred default for ordinary modern RTL, including outputs assigned in `always_ff` blocks. The explicit `wire` type remains useful when the design must emphasize net connectivity, model tri-state behavior, or intentionally use resolved multiple drivers.

Simulation Constructs That Are Generally Not Synthesizable

Simulation supports several constructs that describe a test environment rather than physical RTL hardware. Their observed purposes and synthesis limitations are summarized below. Exact support can vary by synthesis tool and target technology; for example, some FPGA tools allow restricted initialization.

Construct	Simulation use	Reason it is generally not synthesizable
Delay controls (`#10`, `#(period)`)	Schedule stimulus or events after a specified simulation time.	Physical delays are determined by cells and routing. A time delay in RTL does not directly specify clocked storage or a realizable delay structure.
`initial` blocks	Initialize a testbench, generate vectors, or start a simulation process.	ASIC state is normally established with reset circuitry. Restricted initialization may be supported for some FPGA resources, but testbench-style processes are not hardware.
`real` data type	Perform convenient floating-point calculations in a model or testbench.	A generic simulation `real` does not directly define a fixed hardware representation or floating-point architecture. Synthesizable arithmetic must use a supported bit-level representation or suitable IP.
`$display`, `$monitor`	Print values and debugging messages to the simulator console.	Console output is a simulator service and has no direct gate-level hardware equivalent.
File I/O (`$fopen`, `$fwrite`, `$fclose`)	Read test vectors or write simulation results and logs.	Synthesized logic has no implicit operating-system file system. External data access requires an explicitly designed memory or communication interface.

Conclusion

This experiment established the fundamental relationship between SystemVerilog RTL and its eventual hardware implementation. The RTL-to-transistor flow was examined from elaboration and logic optimization through technology mapping and physical design. The annotated 8-bit counter demonstrated sequential logic, flip-flop inference, asynchronous reset, synchronous enable control, state retention, and natural overflow, while the two-input AND gate demonstrated combinational logic through continuous assignment. The observed behavior of both modules agreed with their intended functions. The experiment also clarified that `logic` is the preferred type for most modern RTL signals, whereas `wire` remains useful for explicit net connectivity, tri-state structures, and resolved multiple-driver applications. Finally, delay controls, testbench-style `initial` blocks, `real` variables, console system tasks, and file I/O were identified as constructs used mainly for simulation because they do not directly describe realizable hardware. Overall, the work provided a practical foundation for writing clear, synthesizable RTL and distinguishing hardware design code from simulation-only verification code.

Day 2: State Machine Design & Analysis

Objective

The objective of this experiment was to design and analyze a finite-state machine (FSM) that detects the serial bit sequence `1011`. The detector was required to support overlapping occurrences, produce a clear state diagram and transition table, and be expressed as synthesizable SystemVerilog RTL.

Procedure

Overlapping `1011` Sequence Detector

The detector examines one input bit, `data_in`, on each positive clock edge. It recognizes the sequence `1, 0, 1, 1`. It is an *overlapping* detector, meaning that bits already received may form the beginning of the next valid sequence. For example, the stream `1011011` contains two overlapping occurrences of `1011`: one ending at bit 4 and another ending at bit 7. The fourth bit of the first occurrence is reused as the first bit of the second occurrence.

A Moore-machine structure was selected. Therefore, the detection output depends only on the current state and is asserted while the FSM occupies state `S4`.

State Definitions

IDLE (S0): Initial state. No useful prefix has been retained, and the detector is waiting for the first `1`.

S1 — detected `1`: The received suffix matches the first bit of the required sequence.

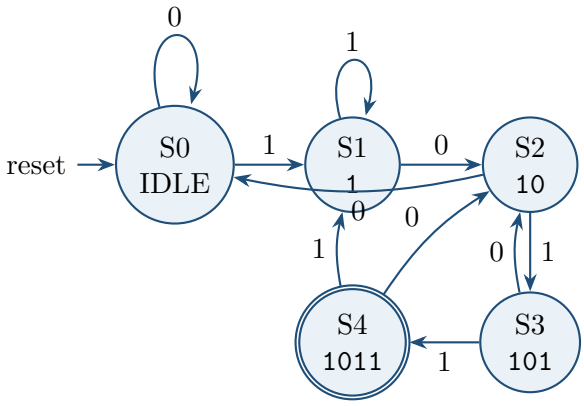
S2 — detected `10`: The two most recent useful bits match the prefix `10`.

S3 — detected `101`: The received suffix matches the first three bits of the sequence. One additional `1` completes a detection.

S4 — detected `1011`: The complete sequence has been recognized, so `detected` is asserted. The next transition preserves any useful suffix to permit overlapping detections.

State Diagram

The transition labels show the value of `data_in`. State `S4` is drawn as an accepting state because its output is `detected = 1`.



The return paths are essential for overlap handling. In particular, `S4` followed by input `0` moves to `S2`, because the suffix `10` is already a valid prefix of a new sequence. Input `1` moves to `S1`, retaining that bit as a possible new start.

State-Transition Table

Current state	Meaning	Next state for 0	Next state for 1
S0	No matched prefix	S0	S1
S1	`1`	S2	S1
S2	`10`	S0	S3
S3	`101`	S2	S4
S4	`1011` detected	S2	S1

Moore vs. Mealy State Machines: A Comparison

In the context of finite state machines (FSMs), the fundamental distinction between Moore and Mealy lies in how the output is generated. In a Moore machine, the output is a function solely of the current state. This makes them inherently more stable and simpler to design because the output only changes synchronously with the state transition on a clock edge. There is no risk of glitches on the output due to rapid input changes. Conversely, in a Mealy machine, the output is a function of both the current state AND the current inputs. This allows the Mealy machine to react to inputs faster (it can produce an output in the same clock cycle as the input arrives) and often requires fewer states than a Moore equivalent. However, this speed comes at the cost of potential glitches or hazards in the output combinational logic path.

Example of Choice:

Choose Moore: For a Traffic Light Controller. The lights (outputs) should only change at specific, clocked intervals. The stability of Moore makes it perfect for controlling physical outputs that must change cleanly.

Choose Mealy: For a Sequence Detector with a faster response. If a detector needs to output a high pulse immediately upon detecting the last bit of a sequence (within the same clock cycle), a Mealy machine can achieve this with one less state than a Moore machine, providing a speed advantage.

SystemVerilog Implementation

The FSM was implemented using separate sequential and combinational blocks. The state register is updated by `always_ff`, while `always_comb` calculates the next state. The output is decoded from the registered current state, consistent with a Moore machine. Codes are provides in the github. **Github link:**

Observations and Results

Transition Analysis

Resetting the design placed the FSM in `S0`. A zero in this state was ignored, while a one advanced the detector to `S1`. Each subsequent matching input advanced the FSM through `S2` and `S3` until the final one moved it into `S4`. The `detected` output was therefore asserted only after all four required bits had been sampled.

The fallback transitions retained the longest suffix that could also be a prefix of `1011`. For example, receiving zero in `S3` produced the suffix `10`, so the FSM returned to `S2` instead of `S0`. This avoided discarding useful input history.

Overlapping Input Trace

The input stream `1011011` verified the overlapping behavior. Starting from `S0`, the state sequence and output were as follows.

Bit position	Input	State after clock edge	detected
Reset	–	S0	0
1	1	S1	0
2	0	S2	0
3	1	S3	0
4	1	S4	1
5	0	S2	0
6	1	S3	0
7	1	S4	1

The output pulses at bit positions 4 and 7 confirm two valid detections. The transition from `S4` to `S2` at bit 5 demonstrates that the final `1` of the first occurrence was retained as the first bit of the next overlapping occurrence.

Expected Hardware

Synthesis of the RTL is expected to infer a three-bit state register, next-state combinational logic, reset logic, and a state decoder for `detected`. The enumerated state type improves readability while allowing the synthesis tool to choose or optimize the actual state encoding according to its settings. The complete assignment of `next_state` in every combinational path prevents unintended latch inference.

Conclusion

An overlapping Moore finite-state machine for detecting `1011` was successfully designed and analyzed. Five states were used to represent the progressively matched prefixes of the target sequence, and fallback transitions were selected to preserve useful suffixes rather than restart unnecessarily. The transition table, state diagram, and SystemVerilog implementation describe the same behavior. Analysis of the input `1011011` produced detection pulses after bits 4 and 7, confirming correct overlap handling. The experiment demonstrated how a serial pattern-recognition problem can be translated into a clear state model and then into structured, synthesizable RTL without inferring unintended storage.

Day 3: The Verification Mindset & Testbench Architecture

Objective

The objective of this experiment was to understand why design verification is treated as a discipline separate from RTL design, identify the major components and information flow of a reusable testbench, develop an initial FIFO verification plan, and define behavioral assertions for the overlapping `1011` sequence detector designed on Day 2.

Procedure

Why Verification Is Separate from RTL Design

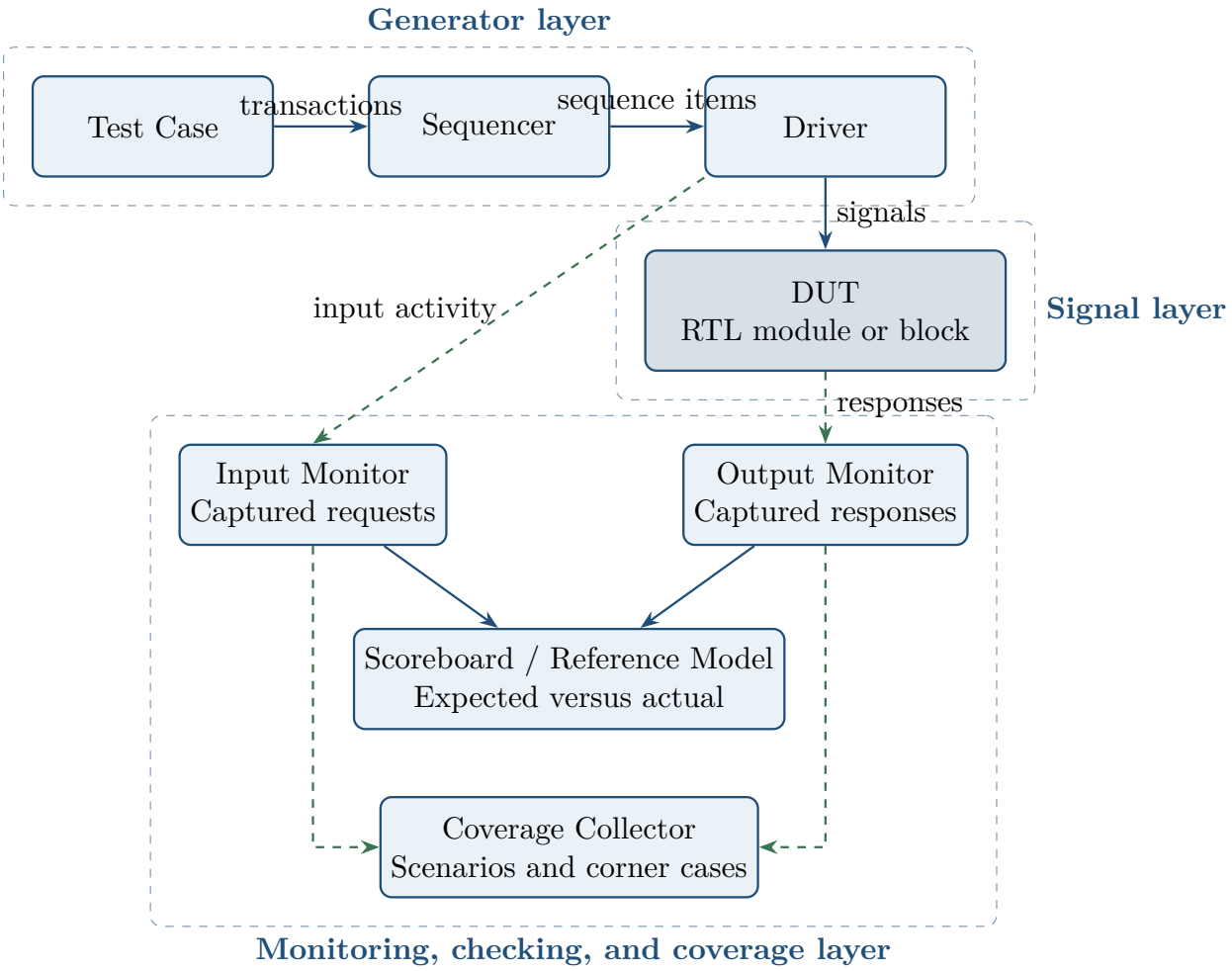
RTL design and design verification contribute to the same chip-development process but pursue different immediate goals. The RTL designer is primarily a creator: architectural requirements must be translated into correct, synthesizable hardware while satisfying performance, power, and area targets. This requires attention to datapaths, control logic, timing, resource use, and physical implementability.

The design-verification (DV) engineer adopts an intentionally adversarial mindset. Instead of asking only how the circuit should work, the verifier asks how it might fail. This includes malformed transactions, reset interactions, boundary values, illegal sequences, simultaneous events, long random runs, and corner cases that may not have been considered during implementation. The DV engineer translates the specification into tests, assertions, reference models, scoreboards, and coverage goals that provide measurable evidence of correctness.

Designers still perform valuable unit-level checks on their own RTL; however, independent verification reduces the risk of common assumptions affecting both the implementation and its tests. A separate DV perspective is therefore an important defense against bugs escaping to silicon, where a correction may require an expensive re-spin and cause a major schedule delay.

Anatomy of a Testbench

A testbench surrounds the Design Under Test (DUT), generates legal and illegal stimulus, observes interface activity, predicts expected behavior, checks the actual response, and measures which scenarios have been exercised. The diagram below shows a standard transaction-oriented architecture similar to that used in a UVM environment.



The forward path carries stimulus from the test to the sequencer and driver. The driver converts transaction-level requests into clock-by-clock signal activity at the DUT interface. Monitors passively reconstruct transactions from the observed signals. Those transactions are then used by the scoreboard for checking and by the coverage collector for measuring verification progress.

Responsibilities of the Testbench Components

Component	Primary responsibility
Test case	Selects and configures a scenario such as reset, random traffic, an error case, or a boundary-condition test.
Sequencer	Controls the order of generated transactions and supplies them to the driver.
Driver / BFM	Converts high-level transactions into the pin-level protocol understood by the DUT.
Monitor	Passively samples interface signals and converts observed activity back into transaction objects without judging correctness.
Scoreboard	Uses a reference model or algorithm to predict expected behavior and compares it with the DUT's observed output.
Coverage collector	Records whether required functions, values, combinations, and corner cases have been exercised.

FIFO Feature-Verification Preview

An initial verification plan was prepared for a future First-In-First-Out (FIFO) buffer. Each feature was converted into a testable requirement with a clear expected result.

Feature	Verification test
Write operation	Assert `wr_en` while the FIFO is not full and verify that `wr_data` is stored at the next write location and can later be read in the correct order.
Read operation	Assert `rd_en` while the FIFO is not empty and verify that the oldest stored word appears on `rd_data` and the read position advances correctly.
Full-flag behavior	Fill the FIFO to its declared capacity and verify that `full` asserts immediately after the capacity-filling write and stays asserted until a valid read creates space.
Empty-flag behavior	Read the last valid entry and verify that `empty` asserts immediately afterward and remains asserted until a valid write adds an entry.
Overflow protection	Assert `wr_en` while `full` is high and verify that no stored entry is overwritten, the write position does not advance, and `full` remains asserted.

This plan establishes what must be checked before implementation details are allowed to influence the tests. Additional FIFO verification would normally cover underflow, simultaneous read and write operations, reset, pointer wraparound, data ordering, and parameterized widths and depths.

Plain-English Assertions for the Sequence Detector

Assertions describe properties that must hold continuously during simulation or formal analysis. The following properties were defined for the Day 2 Moore detector.

1. **Detection-pulse stability:** If `detected` is high on a sampled clock edge, it must be low on

the next sampled edge. The detection state `S4_1011` always transitions to a non-detection state, so two consecutive high cycles are illegal.

2. **Reset behavior:** Whenever `rst_n` is low, the FSM must be in its idle state and `detected` must be low. After reset release, detection may occur only after a complete valid input sequence has been sampled.

3. **Output-state dependency:** The `detected` output may be high only when `current_state` equals `S4_1011`. Conversely, occupying `S4_1011` must assert `detected`. This verifies the defining Moore-machine relationship between state and output.

The state name in the third property is deliberately matched to the Day 2 RTL: `S4_1011` is the state representing the complete sequence, whereas `S3_101` represents only the partial prefix `101`.

Observations and Results

Monitor and Scoreboard Comparison

The monitor and scoreboard were found to be complementary rather than interchangeable. The monitor answers the question, "What happened at the interface?" The scoreboard answers, "Was that behavior correct?" Keeping observation separate from checking improves reuse because the same monitor can supply transactions to scoreboards, coverage collectors, protocol checkers, and debug logs.

Characteristic	Monitor	Scoreboard
Role	Passive observer	Active correctness checker
Input	Clocked DUT interface signals	Transactions reported by monitors
Main operation	Samples and reconstructs transactions	Predicts expected results and compares them with actual results
Design knowledge	Protocol and sampling rules	Functional behavior and reference-model rules
Output	Observed transaction stream	Pass, mismatch, or diagnostic result

Concrete FIFO Checking Example

For FIFO verification, an input monitor observes `wr_en`, `wr_data`, and `rd_en`, while an output monitor captures valid `rd_data` responses and status flags. The scoreboard maintains an internal software queue as the reference model. Each accepted write pushes the observed data into this queue. Each accepted read pops the oldest expected word and compares it with the actual word reported by the output monitor. A mismatch indicates an error in storage, ordering, pointer control, or interface timing.

This example also demonstrates why a monitor must not perform the comparison itself: signal observation is protocol-specific, whereas the queue model and expected behavior belong to the checking policy. Separating them allows either part to be replaced or reused independently.

Verification Outcomes

The exercise produced four verification artifacts before running a simulation: a component-level testbench architecture, a feature-based FIFO test plan, three sequence-detector properties, and a reference-model strategy for FIFO checking. These artifacts convert informal design intent into observable stimulus, explicit expected results, and measurable coverage targets. They also reveal missing cases early; for example, the five requested FIFO features naturally expose the need for later tests of underflow and simultaneous operations.

Conclusion

This experiment established verification as an independent, specification-led engineering activity rather than a final check performed after RTL coding. A structured testbench was decomposed into tests, a sequencer, driver, monitors, scoreboard, reference model, and coverage collector, with each component given a distinct responsibility. The FIFO preview showed how design features can be translated into precise tests for data transfer, status flags, and overflow protection. Assertions for the `1011` detector demonstrated how expected temporal and state-dependent behavior can be checked continuously. Finally, the monitor-scoreboard comparison showed that reliable verification requires both accurate observation and an independent prediction of correct behavior. These principles provide the foundation for scalable, reusable verification environments and reduce the risk of costly functional bugs escaping into silicon.

Day 4: RTL Implementation & Initial Simulation

Objective

The objective of this experiment was to convert the overlapping `1011` sequence-detector specification into a precise RTL-ready design, define a smoke-test strategy, and predict the cycle-by-cycle behavior before examining simulation results. Particular attention was given to the consistency between the state encoding, transition table, output logic, and overlap requirement.

Procedure

Mini-Specification: Overlapping `1011` Detector

Module name: `seq_detect_1011`

Port	Description
`clk`	Clock input. All sequential state updates occur on the positive edge.
`rst_n`	Active-low asynchronous reset. When asserted, it returns the FSM to `IDLE`.
`din`	Serial input bit examined for the target sequence.
`detected`	Single-bit output asserted for one clock cycle when the overlapping sequence `1011` is recognized.

State Encoding

The compact detector uses four states to record partial progress through the target pattern.

State	Encoding	Description
IDLE	`2'b00`	Waiting for the first `1`.
S1	`2'b01`	The useful suffix is `1`.
S10	`2'b10`	The useful suffix is `10`.
S101	`2'b11`	The useful suffix is `101`.

Although the original mini-specification labeled this structure as a Moore FSM, the four-state organization is a *Mealy-style* detector: the output is asserted when the current state is `S101` *and* the current input is `1`. A strict Moore implementation would require a separate fifth detection state or a registered output state. The four-state Mealy structure was retained here because it matches the supplied two-bit encoding and produces the pulse as the final sequence bit is sampled.

Corrected Transition Table

The table below implements genuine overlap behavior. After detecting `1011`, the FSM moves to `S1`, not `IDLE`, because the final `1` may also be the first bit of a new occurrence.

Current state	din	Next state	detected	Reason
IDLE	0	IDLE	0	No useful prefix
IDLE	1	S1	0	First `1` found
S1	0	S10	0	Prefix `10` found
S1	1	S1	0	New leading `1` retained
S10	0	IDLE	0	No useful suffix
S10	1	S101	0	Prefix `101` found
S101	0	S10	0	Suffix `10` retained
S101	1	S1	1	`1011` found; final `1` retained

The output equation implied by this table is

$$\texttt{detected} = (\texttt{current\_state} = \texttt{S101}) \wedge \texttt{din}.$$

This direct dependence on both state and input confirms the Mealy classification.

RTL Implementation

The RTL uses a two-bit state register with active-low asynchronous reset, a combinational next-state block implementing the corrected transition table, and combinational detection logic for the final `1`. Complete source code for the detector is provided in the project repository.

GitHub link: _____

Smoke Testbench

The smoke testbench provides a clock, applies and releases reset, drives a small set of directed serial streams, and observes `detected`. At minimum, the initial simulation should check the following cases:

1. reset forces the FSM to `IDLE` and clears `detected`;
2. a stream without `1011` produces no detection pulse;
3. a single `1011` produces exactly one pulse on its final bit;
4. `1011011` produces two pulses and confirms overlap handling;
5. additional leading ones and partial patterns do not create false detections.

The RTL and smoke-testbench source files are provided in the GitHub repository.

GitHub link: _____

Hand-Trace Prediction

Before simulation, the input stream `1 1 0 1 0 1 1` was traced through the corrected RTL logic. The output shown for each cycle is the combinational Mealy result produced from the state before the edge and the input applied for that cycle.

Cycle	din	State before	State after	detected	Comment
Reset	X	IDLE	IDLE	0	Initialized by reset
1	1	IDLE	S1	0	First `1` retained
2	1	S1	S1	0	Latest `1` remains a valid start
3	0	S1	S10	0	Prefix `10` formed
4	1	S10	S101	0	Prefix `101` formed
5	0	S101	S10	0	No detection; suffix `10` retained
6	1	S10	S101	0	Prefix `101` formed again
7	1	S101	S1	1	`1011` detected; overlap retained

Therefore, the predicted `detected` signal is high only on cycle 7 for this input stream. Cycle 5 cannot assert detection because its input is `0`; the four bits ending there are `1010`, not `1011`.

Observations and Results

Specification Consistency Check

The hand analysis exposed two inconsistencies in the initial description. First, a detector with four progress states and an output dependent on `S101` plus `din` is Mealy-style rather than Moore-style. Second, returning to `IDLE` after a detection discards the final `1` and breaks the required overlap behavior. Returning to `S1` resolves this problem without adding another state.

Expected Initial-Simulation Result

For the directed input `1101011`, the waveform should show reset placing the state at `IDLE`, followed by the sequence

`S1, S1, S10, S101, S10, S101, S1`.

The `detected` waveform should remain low during cycles 1–6 and pulse high during cycle 7. A simulator result that pulses on cycle 5 would indicate an error in the stimulus alignment, output sampling, or implementation because the input at that cycle is zero.

The separate overlap smoke test `1011011` should produce pulses on its fourth and seventh bits. This test directly distinguishes the corrected `S101` to `S1` transition from the non-overlapping transition to `IDLE`.

Conclusion

The Day 4 exercise converted the `1011` detector into an RTL-ready four-state specification and established a focused smoke-test strategy. Careful hand tracing showed that the compact two-bit design is a Mealy detector, because `detected` depends on both `S101` and `din`. Correct overlap handling requires the detection transition to retain the final `1` by moving to `S1`. For the input `1101011`, only one valid occurrence exists, so the predicted output pulse occurs on cycle 7. This analysis demonstrates the value of checking specifications manually before simulation: inconsistencies in FSM classification, transitions, and expected waveforms can be corrected before they become RTL or testbench bugs.

Day 5: Traffic Light Controller — State Machine with Timing

Objective

The objective of this experiment was to design and initially verify a timed finite-state machine for a four-way intersection consisting of a main road and a side road. The controller was required to follow a safe four-state sequence, hold each state for a parameterized number of clock cycles, generate the correct three-bit light outputs, and recover predictably from reset.

Procedure

State Definitions

The controller cycles continuously through four traffic phases.

MAIN_GREEN: The main road is green while the side road is red.

MAIN_YELLOW: The main road is yellow while the side road remains red.

SIDE_GREEN: The side road is green while the main road is red.

SIDE_YELLOW: The side road is yellow while the main road remains red.

The three-bit light encoding used in the expected log is `001 = green`, `010 = yellow`, and `100 = red`. At no time may both roads display green or yellow simultaneously.

Duration Parameters

Short durations were selected to make the initial simulation compact. In an implementation intended for physical traffic control, these values would be scaled from the system clock or driven by a lower-frequency timing tick.

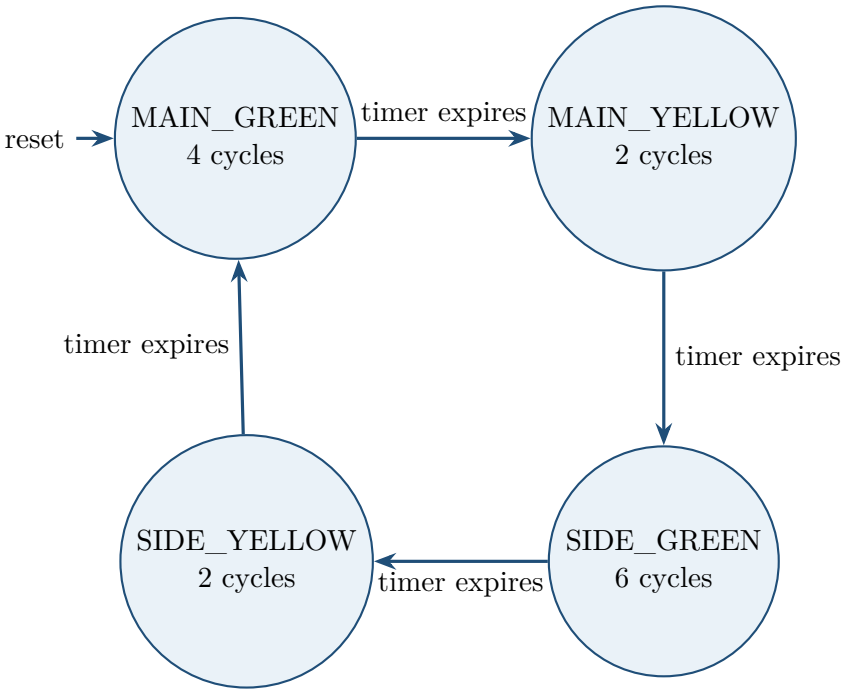
Parameter	Cycles	Description
`MAIN_GREEN_DURATION`	4	Main-road green interval
`MAIN_YELLOW_DURATION`	2	Main-road yellow interval
`SIDE_GREEN_DURATION`	6	Side-road green interval
`SIDE_YELLOW_DURATION`	2	Side-road yellow interval

State and Duration Table

State	Duration	Main road	Side road
MAIN_GREEN	4 cycles	Green	Red
MAIN_YELLOW	2 cycles	Yellow	Red
SIDE_GREEN	6 cycles	Red	Green
SIDE_YELLOW	2 cycles	Red	Yellow

State Sequence

The controller follows a fixed cyclic order. Reset initializes the machine to `MAIN_GREEN`; after each state's timer expires, the next phase begins and its duration counter is loaded.



For a state whose duration is D cycles, the down-counter is loaded with $D - 1$. While the counter is greater than zero, it decrements and the state is held. When the counter is zero, the controller transitions on the next active clock edge and loads the next state's counter with its duration minus one. This convention produces exactly D observed cycles in each state.

RTL Implementation

The RTL implementation contains a state register, a duration down-counter, combinational next-state and next-counter logic, and Moore-style output decoding from the current state. An active-low

asynchronous reset places the controller in `MAIN_GREEN`. Its counter is initialized to

$$\texttt{MAIN\_GREEN\_DURATION} - 1.$$

The complete RTL source is provided in the project repository.

GitHub link: _____

Smoke Testbench

The smoke testbench generates the clock, applies reset, and runs the controller for more than one complete 14-cycle traffic-light period. It should perform or enable checks for the following behaviors:

1. reset selects `MAIN_GREEN`, sets the safe light outputs, and loads the main-green timer;
2. states occur only in the order `MAIN_GREEN`, `MAIN_YELLOW`, `SIDE_GREEN`, `SIDE_YELLOW`;
3. each state remains active for its parameterized number of cycles;
4. the counter decreases to zero without underflow or stalling;
5. the output encodings match the current state and never allow two non-red directions simultaneously;
6. the sequence repeats correctly after `SIDE_YELLOW`.

The RTL and smoke-testbench source files are provided in the GitHub repository.

GitHub link: _____

Expected Simulation Log

The following transcript represents the expected state, output, and timer values at successive rising clock edges. The leading values `15, 25, 35, ...` are simulator timestamps separated by one clock period; they are retained from the proposed log even though its label reads "Cycle."

```
Cycle:  15 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 3
Cycle:  25 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 2
Cycle:  35 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 1
Cycle:  45 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 0
Cycle:  55 | State: MAIN_YELLOW | Main: 010 | Side: 100 | Counter: 1
Cycle:  65 | State: MAIN_YELLOW | Main: 010 | Side: 100 | Counter: 0
Cycle:  75 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 5
Cycle:  85 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 4
Cycle:  95 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 3
Cycle: 105 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 2
Cycle: 115 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 1
Cycle: 125 | State: SIDE_GREEN  | Main: 100 | Side: 001 | Counter: 0
Cycle: 135 | State: SIDE_YELLOW | Main: 100 | Side: 010 | Counter: 1
Cycle: 145 | State: SIDE_YELLOW | Main: 100 | Side: 010 | Counter: 0
Cycle: 155 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 3
Cycle: 165 | State: MAIN_GREEN  | Main: 001 | Side: 100 | Counter: 2
```

Observations and Results

Timing Analysis

The expected transcript contains four consecutive `MAIN_GREEN` samples, two `MAIN_YELLOW` samples, six `SIDE_GREEN` samples, and two `SIDE_YELLOW` samples. The complete cycle therefore contains

$$4 + 2 + 6 + 2 = 14 \text{ clock cycles.}$$

At timestamp 155, the controller returns to `MAIN_GREEN`, confirming the start of the next period. Each state begins with a counter value of one less than its duration and remains active through the sample at counter zero.

The light values also remain mutually safe throughout the log. During a green or yellow phase for one road, the other road is always encoded as red (`100`). No entry permits both roads to display a non-red indication.

Issue Log — Day 5

Bug ID	Date	Summary	Status
BUG-003	Day 5	Duration counter remained at its maximum value or failed to cause a state transition.	Fixed
BUG-004	Day 5	Light outputs did not update correctly when the FSM state changed.	Fixed

BUG-003 — Counter decrement and terminal-count handling. The next-counter logic did not distinguish a positive counter from the terminal value zero correctly. This could leave the timer at zero and prevent the FSM from advancing, or cause an unintended unsigned underflow. The logic was restructured so that `counter > 0` causes a decrement, whereas `counter == 0` causes a state transition and loads the next state's duration. Default assignments for the next state and next counter were also added to prevent unintended latches.

BUG-004 — Output update on state transition. The light-decoding logic used an incomplete or incorrect sensitivity mechanism, so changes of state were not always reflected at the outputs. The decoder was reimplemented using `always_comb`, with safe default assignments and a complete case for every legal state. This makes the light outputs a purely combinational function of the registered current state and ensures immediate, deterministic updates after each state transition.

Expected Verification Outcome

After the two fixes, the expected smoke-test log matches the specified state durations and output encodings. The counter terminates cleanly at zero, every transition loads the correct next duration, and the light decoder follows the state without retaining stale values. A complete simulation should additionally use assertions to verify legal state transitions, timer range, mutually exclusive green lights, and known outputs after reset.

Conclusion

The timed traffic-light controller was specified as a four-state Moore FSM with parameterized durations for the main and side roads. A duration-minus-one down-counter provides exact state holding times while allowing a transition when the terminal count is reached. The expected simulation log demonstrates the required 4–2–6–2 cycle pattern, correct three-bit light encodings, safe mutual exclusion, and repetition after a 14-cycle period. Analysis of BUG-003 and BUG-004 emphasized the importance of explicit terminal-count handling, complete combinational assignments, and `always_comb` output decoding. Together, the state machine, timer, smoke-test plan, and issue log provide a clear basis for implementing and verifying a safe synchronous traffic-light controller.

Day 6: Synchronous FIFO — Memory Buffer Implementation

Objective

The objective of this experiment was to specify and initially verify a parameterized synchronous First-In-First-Out (FIFO) memory buffer. The work focused on correct data ordering, occupancy tracking, full and empty flag generation, pointer wraparound, blocked overflow and underflow operations, and simultaneous read/write behavior.

Procedure

Module Definition and Parameters

The FIFO stores data words in the order in which they are accepted. Both read and write operations use the same clock, so the design is a synchronous FIFO.

Module name: `sync_fifo`

Parameter	Default	Description
`WIDTH`	8	Number of bits in each stored data word
`DEPTH`	8	Number of words that can be stored

The write and read pointers require enough bits to address every memory entry, while the occupancy counter must represent values from zero through `DEPTH`. Typical derived widths are

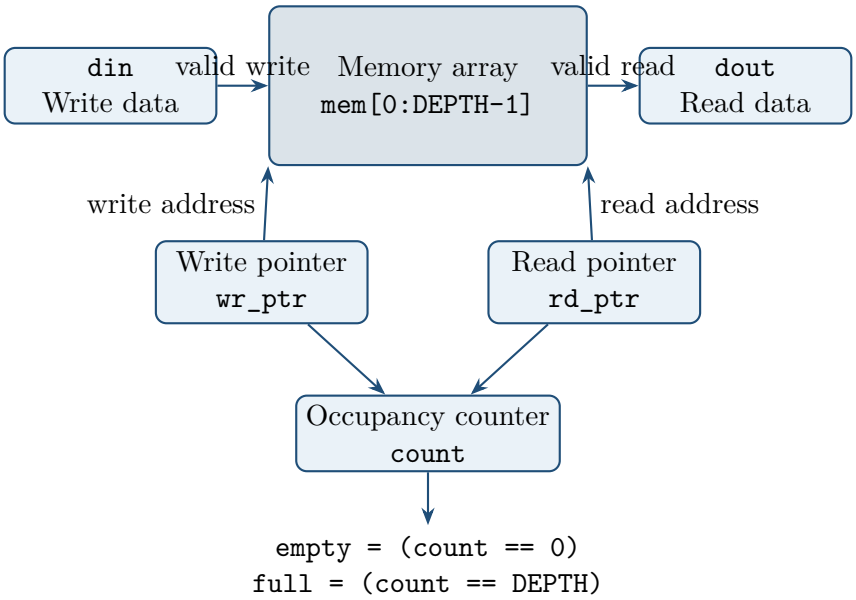
$$\texttt{PTR\_WIDTH} = \lceil \log_2(\texttt{DEPTH}) \rceil, \qquad \texttt{COUNT\_WIDTH} = \lceil \log_2(\texttt{DEPTH} + 1) \rceil.$$

Port Definitions

Port	Direction	Width	Description
`clk`	Input	1	System clock
`rst_n`	Input	1	Active-low asynchronous reset
`wr_en`	Input	1	Requests a write into the FIFO
`rd_en`	Input	1	Requests removal of the oldest word
`din`	Input	WIDTH	Data word presented for writing
`dout`	Output	WIDTH	Registered data word produced by a valid read
`full`	Output	1	High when all `DEPTH` entries are valid
`empty`	Output	1	High when no valid entries are stored

FIFO Architecture

The memory array is addressed independently by the write and read pointers. The occupancy counter records the number of valid entries and supplies the full and empty status conditions.



A write is accepted only when `wr_en` is high and the FIFO is not full. A read is accepted only when `rd_en` is high and the FIFO is not empty. Defining these accepted operations explicitly avoids ambiguity:

$$\texttt{write\_valid} = \texttt{wr\_en} \wedge \neg\texttt{full}, \qquad \texttt{read\_valid} = \texttt{rd\_en} \wedge \neg\texttt{empty}.$$

RTL Implementation

The RTL contains a `WIDTH`-bit by `DEPTH`-entry memory array, write and read pointers, an occupancy counter, and status logic. On each positive clock edge, accepted writes update the memory and write pointer; accepted reads update `dout` and the read pointer. The occupancy counter increments for write-only activity, decrements for read-only activity, and remains unchanged when both operations are accepted together. Complete RTL is provided in the project repository.

GitHub link: _____

Smoke Testbench

The smoke testbench should reset the FIFO, fill it to capacity, attempt an overflow write, empty it in order, attempt an underflow read, and exercise simultaneous read and write traffic. Its minimum checks are:

1. reset clears both pointers and `count`, asserts `empty`, and deasserts `full`;
2. eight accepted writes increase `count` from 0 to 8 and assert `full`;
3. a ninth write while full is blocked and does not overwrite data;
4. eight accepted reads reproduce the written words in FIFO order and return `count` to zero;
5. a read while empty is blocked and leaves `dout` unchanged;
6. simultaneous accepted read and write operations preserve `count` while advancing both pointers.

The RTL and smoke-testbench source files are provided in the GitHub repository.

GitHub link: _____

Expected Simulation Log

The proposed transcript is shown below. As in Day 5, the leading values are simulation timestamps at successive clock edges rather than literal ordinal cycle numbers.

```
Cycle:  15 | wr=0 rd=0 din=  0 dout=  0 count=0 full=0 empty=1
Cycle:  25 | wr=1 rd=0 din=  0 dout=  0 count=1 full=0 empty=0
Cycle:  35 | wr=1 rd=0 din= 16 dout=  0 count=2 full=0 empty=0
Cycle:  45 | wr=1 rd=0 din= 32 dout=  0 count=3 full=0 empty=0
Cycle:  55 | wr=1 rd=0 din= 48 dout=  0 count=4 full=0 empty=0
Cycle:  65 | wr=1 rd=0 din= 64 dout=  0 count=5 full=0 empty=0
Cycle:  75 | wr=1 rd=0 din= 80 dout=  0 count=6 full=0 empty=0
Cycle:  85 | wr=1 rd=0 din= 96 dout=  0 count=7 full=0 empty=0
Cycle:  95 | wr=1 rd=0 din=112 dout=  0 count=8 full=1 empty=0  <-- FIFO full
Cycle: 105 | wr=1 rd=0 din=100 dout=  0 count=8 full=1 empty=0  <-- Write blocked
Cycle: 115 | wr=0 rd=1 din=  0 dout=  0 count=7 full=0 empty=0
Cycle: 125 | wr=0 rd=1 din=  0 dout= 16 count=6 full=0 empty=0
Cycle: 135 | wr=0 rd=1 din=  0 dout= 32 count=5 full=0 empty=0
Cycle: 145 | wr=0 rd=1 din=  0 dout= 48 count=4 full=0 empty=0
Cycle: 155 | wr=0 rd=1 din=  0 dout= 64 count=3 full=0 empty=0
Cycle: 165 | wr=0 rd=1 din=  0 dout= 80 count=2 full=0 empty=0
Cycle: 175 | wr=0 rd=1 din=  0 dout= 96 count=1 full=0 empty=0
Cycle: 185 | wr=0 rd=1 din=  0 dout=112 count=0 full=0 empty=1  <-- FIFO empty
Cycle: 195 | wr=0 rd=1 din=  0 dout=112 count=0 full=0 empty=1  <-- Read blocked
```

Observations and Results

Simulation-Log Analysis

The expected log begins with an empty FIFO. Eight valid writes store the values `0, 16, 32, 48, 64, 80, 96, 112`, increasing `count` to eight and asserting `full`. The attempted write of `100` is blocked, so it does not appear in the later read sequence. This confirms overflow protection.

The subsequent reads return the original values in insertion order. After the eighth read, `count` becomes zero and `empty` asserts. The final read request is blocked, and `dout` retains the stale value `112`. This retained output is not a valid new FIFO word; consumers must qualify it with a successful read condition or the `empty` flag.

Knowledge Base: FIFO Flags and Pointer Management

Full and empty computation. An explicit occupancy counter makes the status equations direct and avoids the ambiguity that equal binary read and write pointers can represent either an empty or full FIFO:

$$\texttt{full} = (\texttt{count} = \texttt{DEPTH}), \qquad \texttt{empty} = (\texttt{count} = 0).$$

The counter must be wide enough to represent `DEPTH`; for a depth of eight, four bits are required to represent values 0 through 8.

Pointer wraparound. Explicit terminal comparison supports both power-of-two and non-power-of-two depths:

```
if (wr_ptr == DEPTH - 1)
    wr_ptr <= '0;
else
    wr_ptr <= wr_ptr + 1'b1;
```

The read pointer uses the same rule. Relying only on natural binary overflow is safe for a power-of-two depth when the pointer width is chosen appropriately, but explicit wrap logic is more generally parameterizable.

Corner Cases

Condition	Required behavior
Simultaneous read and write	If neither boundary blocks an operation, both pointers advance, one word enters as one leaves, and `count` remains unchanged.
Write while full	The write is ignored, memory and write pointer are unchanged, and occupancy remains at `DEPTH`. If a read is also requested, the simple gating policy accepts the read and frees one entry.
Read while empty	The read is ignored, the read pointer and `dout` remain unchanged, and occupancy remains zero. If a write is also requested, the simple gating policy accepts the write and creates one valid entry.
Reset recovery	Both pointers and `count` are cleared. The memory array need not be reset because `count = 0` marks every location as logically invalid. `dout` may be reset separately for deterministic observation.

The occupancy update should use the accepted operations rather than the raw enable inputs:

```
unique case ({write_valid, read_valid})
    2'b10:   count <= count + 1'b1;
    2'b01:   count <= count - 1'b1;
```

```
    2'b11:   count <= count;
    default: count <= count;
endcase
```

Issue Log — Day 6

Bug ID	Date	Summary	Status
BUG-005	Day 6	The full flag remained low after the eighth accepted write because occupancy was not incremented correctly.	Fixed
BUG-006	Day 6	A simultaneous read and write incorrectly decremented the occupancy count.	Fixed

BUG-005 — Full flag did not assert. The count-update conditions did not cover the valid write-only case reliably, so occupancy could fail to reach `DEPTH`. The correction defines `write_valid = wr_en && !full` and `read_valid = rd_en && !empty`, then updates `count` from their two-bit combination. After the eighth accepted write, the count reaches eight and the combinational full comparison asserts.

BUG-006 — Simultaneous operation changed occupancy. An if–else chain gave one operation priority, so both asserted enables could be interpreted as a read-only event. Replacing this logic with an explicit case on `{write_valid, read_valid}` ensures that `2'b11` advances both pointers while holding `count`. At full or empty boundaries, the valid-operation terms correctly indicate whether only one of the two requests can be accepted.

Expected Verification Outcome

After these corrections, the FIFO should fill and empty at the exact occupancy limits, preserve write order, block overflow and underflow, wrap both pointers, and maintain occupancy during simultaneous accepted operations. Assertions can further verify that `count` never exceeds `DEPTH`, never becomes negative, `full` and `empty` are never high together for a positive depth, and blocked operations do not move their associated pointers.

Conclusion

The synchronous FIFO was specified as an eight-entry, eight-bit parameterized memory buffer with independent read and write pointers and an explicit occupancy counter. The expected simulation sequence demonstrates correct filling, full-flag assertion, overflow blocking, ordered reading, empty-flag assertion, and underflow blocking. Explicit pointer wrap logic supports general depths, while accepted-operation terms provide correct counter behavior for write-only, read-only, simultaneous, and boundary cases. The resolution of BUG-005 and BUG-006 showed that FIFO correctness depends on updating occupancy from operations that actually occur rather than from raw enables.

Together, the architecture, smoke-test plan, corner-case rules, and issue analysis provide a sound basis for implementing and verifying a reusable synchronous FIFO.

Day 7: Specification Analysis & Corner-Case Identification

Objective

The objective of this experiment was to study the structure of effective RTL specifications, extract a reusable document format, and identify corner cases for the sequence detector, traffic-light controller, and synchronous FIFO. The resulting material was intended to give RTL and DV engineers a common, unambiguous reference before further implementation and verification.

Procedure

Worked-Example Review: Specification Structure

Review of the worked examples in `example_rtl_and_testplan/` revealed a consistent specification structure. Each section answers a different design or verification question.

Specification section	Purpose
Title and revision history	Records document identity, ownership, versions, dates, and approved changes.
Overview	Explains the design purpose, operating context, and intended use.
Key features	Summarizes the primary capabilities and configurable behavior.
Interface table	Defines every port, direction, width, encoding, and signal meaning.
Microarchitecture	Describes internal blocks, data flow, registers, memories, counters, and block-level relationships.
FSM specification	Defines states, encodings, transitions, outputs, and state diagrams where control logic is present.
Timing	States clocking, reset behavior, latency, throughput, sampling, and cycle-level requirements.
Corner cases	Defines expected behavior at boundaries, during simultaneous events, and under unusual or illegal conditions.
RTL code or reference	Provides the implementation or an exact repository location for the corresponding source.
Verification hints	Identifies important tests, assertions, coverage goals, and likely failure modes.

This format creates a single source of truth shared by design and verification. The interface and timing sections define observable behavior, the microarchitecture and FSM sections guide implementation, and the corner-case and verification sections make the design intent testable.

Corner Cases: Overlapping `1011` Detector

The following cases apply to the compact four-state detector developed on Day 4. Because its output depends on `S101` and `din`, it is Mealy-style rather than a strict Moore FSM.

ID	Scenario	Expected behavior
SEQ-CC-01	Continuous ones, such as `111111`	The FSM remains in `S1` and never asserts `detected`; a zero is required to form the prefix `10`.
SEQ-CC-02	`1011` immediately after reset release	Detection asserts on the fourth sampled bit for one cycle.
SEQ-CC-03	Overlap stream `1011011`	Detection asserts on bits 4 and 7. After each detected final `1`, the FSM returns to `S1` so that bit can begin the next occurrence.
SEQ-CC-04	Reset asserted during a partial match	The asynchronous reset immediately returns the state to `IDLE` and clears detection.
SEQ-CC-05	Long stream without a complete match	The FSM may move among partial-match states but must never generate a false detection.
SEQ-CC-06	Near match `1010`	Detection remains low and the FSM ends in `S10`, retaining the suffix `10`.
SEQ-CC-07	Reset released while `din` is high	The first high input sampled after reset release moves the FSM from `IDLE` to `S1`. Reset assertion remains asynchronously effective.

Corner Cases: Traffic-Light Controller

ID	Scenario	Expected behavior
TLC-CC-01	Reset during any traffic state	The controller immediately returns to `MAIN_GREEN`, selects safe outputs, and reloads the main-green counter.
TLC-CC-02	Duration parameter equals one	The counter loads zero, the state is observed for one cycle, and transition occurs at the following edge.
TLC-CC-03	Large duration near the counter limit	The derived counter width must represent the largest duration minus one without truncation. Invalid parameter combinations should be rejected by an elaboration-time check.
TLC-CC-04	Illegal or unsafe light encoding	Both roads must never show green simultaneously. Every legal state must decode to one active road and a red indication for the other road.
TLC-CC-05	Odd or changed compile-time durations	Any positive values such as 1, 3, 5, and 7 must preserve the correct state order and exact holding times after recompilation.
TLC-CC-06	Transition at terminal count	State and counter change only at the active clock edge; combinational next-state activity must not glitch the registered state or outputs.
TLC-CC-07	Long wraparound simulation	The four-state sequence repeats indefinitely without accumulated timing drift, counter lockup, or illegal transitions.

Corner Cases: Synchronous FIFO

ID	Scenario	Expected behavior
FIFO-CC-01	Read and write requested while full	The write is blocked, the read is accepted, occupancy decreases, and `full` deasserts.
FIFO-CC-02	Read and write requested while empty	The read is blocked, the write is accepted, occupancy increases, and `empty` deasserts.
FIFO-CC-03	Simultaneous operations with one entry	The existing word is read, the new word is written, both pointers advance, and occupancy remains one.
FIFO-CC-04	Both pointers wrap on accepted operations	Each pointer wraps independently at `DEPTH-1`; occupancy and valid-operation logic, not pointer equality alone, determine full and empty status.
FIFO-CC-05	Reset while data is stored	Pointers and count clear, making the FIFO logically empty. Memory bits need not be cleared.
FIFO-CC-06	Write after full assertion	The request is ignored until a valid read creates space; memory and the write pointer remain unchanged.
FIFO-CC-07	Read after empty assertion	The request is ignored until a valid write occurs; the read pointer and `dout` remain unchanged.
FIFO-CC-08	`DEPTH=1`	One write fills the FIFO and one read empties it. Since `$clog2(1)=0`, the RTL must force `PTR_WIDTH` to at least one bit.
FIFO-CC-09	Depth is not a power of two	Explicit comparison with `DEPTH-1` wraps pointers correctly without addressing unused binary pointer values.
FIFO-CC-10	Long write burst without reads	Occupancy rises only to `DEPTH`; all later writes are blocked and `full` remains high.

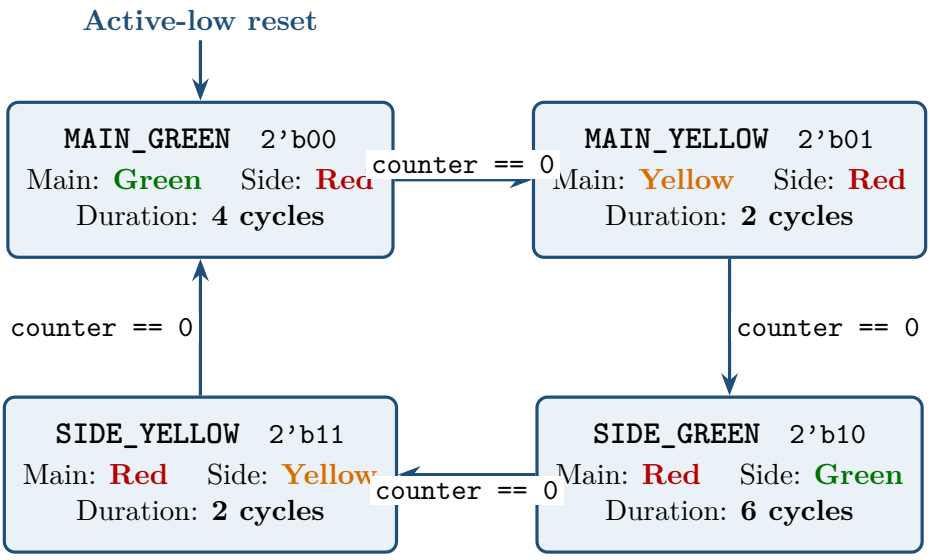
Sequence-Detector Interface Table

Port	Direction	Width	Description
`clk`	Input	1	System clock; sequential state updates occur on its positive edge.
`rst_n`	Input	1	Active-low asynchronous reset that returns the FSM to `IDLE` and suppresses detection.
`din`	Input	1	Serial input bit used to calculate the next state and the Mealy detection condition.
`detected`	Output	1	One-cycle detection pulse asserted when the current state is `S101` and `din` is high.

This interface description intentionally matches the four-state Day 4 RTL. A five-state Moore version would instead define `detected` solely from a registered detection state.

Traffic-Light State Diagram

The diagram below presents the four timed Moore states. Every transition occurs when the registered duration counter reaches zero. Reset returns the controller to the safe initial state, `MAIN_GREEN`.



The three-bit counter width supports values from 0 through 7, which safely covers the largest configured duration of six cycles. The output combinations also enforce mutual exclusion: whenever one road displays green or yellow, the other road remains red.

FIFO Overview for the DV Engineer

The synchronous FIFO is a first-in-first-out memory buffer used to decouple producer and consumer activity within a single clock domain. It stores up to `DEPTH` entries of `WIDTH` bits and supports independent read and write requests in the same cycle. Memory locations are addressed by wrapping read and write pointers, while an occupancy counter determines `full` and `empty`.

From a verification perspective, the most important targets are data ordering, pointer wraparound, occupancy integrity, exact flag timing, simultaneous operations, reset recovery, and non-destructive blocking of writes while full and reads while empty. The reference model can be a software queue: accepted writes push data and accepted reads pop and compare the oldest expected word. This model naturally checks both storage correctness and FIFO ordering.

Observations and Results

Cross-Design Findings

The review showed that the highest-risk specification gaps occur at boundaries: reset release, terminal counters, partial-pattern fallback, simultaneous FIFO operations, full and empty transitions, and parameter extremes. These cases are easy to omit from a basic functional description but often determine whether the RTL works reliably in a real system.

The exercise also showed that interface and behavioral descriptions must agree with the selected microarchitecture. In particular, describing the compact four-state sequence detector as Moore

would conflict with its state-dependent and input-dependent output equation. Resolving this distinction in the specification prevents the RTL, tests, and assertions from implementing different timing expectations.

Verification Priorities

The identified corner cases translate directly into verification tasks. Reset cases should become assertions and directed tests; legal state transitions and traffic-light mutual exclusion should become temporal properties; sequence fallback cases should become directed bit streams; and FIFO boundary and simultaneous-operation cases should be checked against a queue-based scoreboard. Coverage should record every state, transition, flag boundary, pointer wrap, blocked request, and simultaneous-operation combination.

Conclusion

Day 7 established a reusable specification structure and applied it to three different RTL designs. Detailed corner-case analysis exposed the behavioral requirements that are most likely to be misunderstood or omitted, including overlapping pattern retention, one-cycle traffic durations, safe light decoding, FIFO boundary arbitration, and single-entry pointer sizing. The corrected sequence-detector interface and the DV-oriented FIFO overview further demonstrated that a specification must describe observable timing as carefully as basic functionality. A well-structured specification therefore serves not only as design documentation but also as the foundation for test planning, assertions, coverage, and debug.

Day 8: Specification Documentation & Design Cleanup

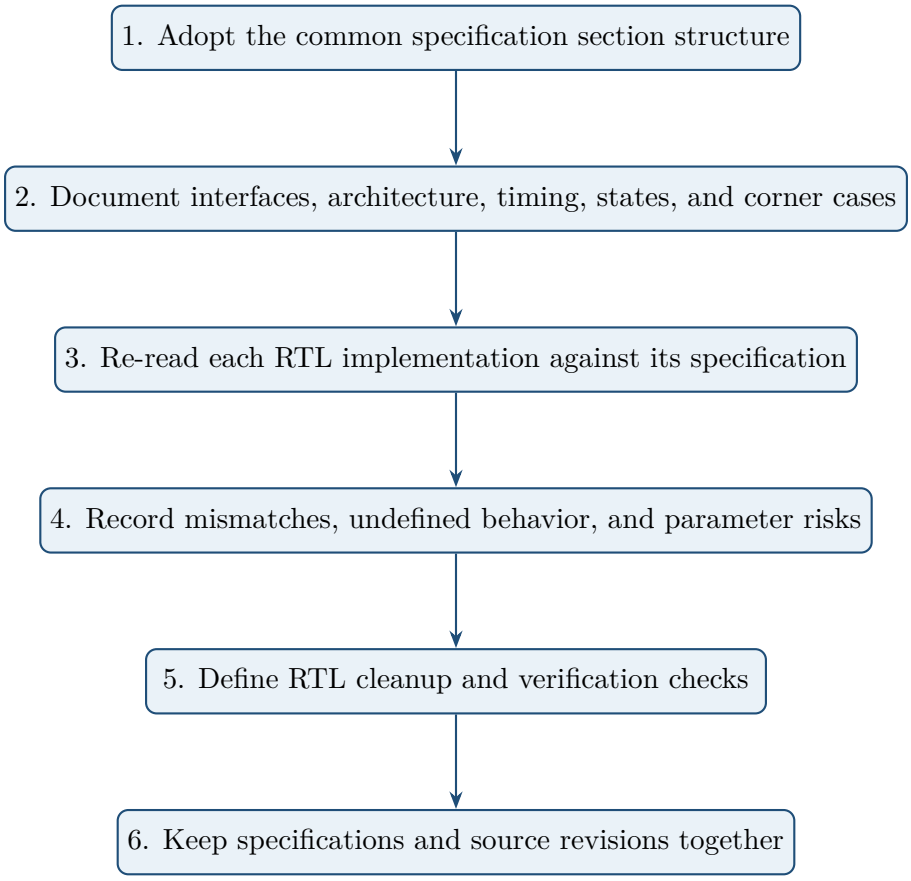
Objective

The objective of this experiment was to consolidate the specifications for all three RTL blocks, compare each implementation against its documented behavior, record inconsistencies, and define cleanup actions that improve synthesizability, parameter safety, reset behavior, and verification clarity.

Procedure

Documentation and Review Workflow

The specification structure identified on Day 7 was applied consistently to the sequence detector, traffic-light controller, and synchronous FIFO. The review process followed the sequence below.



The resulting documentation set uses the same high-level organization for all three blocks while

allowing design-specific sections such as FSM transitions, timed counters, FIFO memory organization, and verification hints.

Experimental Work Recorded

- Applied the worked-example section structure to the three design specifications.
- Completed interface, feature, architecture, timing, corner-case, and verification sections for each RTL block.
- Compared documented behavior with the corresponding RTL intent to find missing defaults, synthesis-only concerns, incomplete validity conditions, undefined read behavior, and hard-coded sizing assumptions.
- Defined cleanup changes so specifications and RTL can be maintained in the same source-control revision.

Inconsistencies and Cleanup Actions

Issue	File	Problem	Cleanup action
Missing default output	`seq_detect_1011.sv`	`detected` was not assigned on every combinational path, risking an unintended latch or unknown value.	Assign `detected = 1'b0` before the state logic and override it only for the valid detection condition.
Unguarded debug function	`traffic_light.sv`	A state-name function used for simulation debug was present in synthesizable RTL.	Surround the debug-only function with `// synthesis translate_off` and `// synthesis translate_on`, or move it into the testbench.
Incomplete validity checks	`sync_fifo.sv`	Occupancy logic used raw enables or incomplete full/empty qualification.	Define separate accepted operations using `wr_en && !full` and `rd_en && !empty`, then update the counter from their combination.
Empty-read output undefined	`sync_fifo.sv`	The expected value of `dout` after a blocked empty read was not documented.	Specify that a blocked read does not update `dout`; it retains the last registered value and is invalid while `empty` is high.
Hardcoded counter width	`traffic_light.sv`	A fixed width of three bits depended on the original duration values.	Derive the width from the largest duration with a minimum width of one and add elaboration-time checks that every duration is positive and representable.

Recommended Cleanup Patterns

Complete combinational defaults. Every `always_comb` block should assign safe defaults before its case or conditional logic. This both documents the normal behavior and prevents latch inference when a state is illegal or an input combination is omitted.

Simulation-only guards. Debug strings, display helpers, and non-synthesizable instrumentation should be kept in the testbench where possible. If a tool-specific debug helper must remain near the RTL, synthesis translation guards clearly separate it from the hardware description.

Derived widths and parameter checks. Widths should be computed from parameters rather than copied from one default configuration. The traffic timer must cover the largest duration minus one, and the FIFO pointer width must remain at least one for `DEPTH=1`. Invalid zero durations, non-positive depths, or widths should be rejected during elaboration rather than producing silently broken hardware.

Documented blocked-operation semantics. The FIFO specification must state not only that an empty read is blocked, but also what happens to every visible output and internal pointer. This closes the gap between "no operation" and the value a testbench or downstream block may still observe on `dout`.

Observations and Results

Design-Quality Improvements

The cleanup review converted several implicit assumptions into explicit design rules. Default assignments make illegal-state behavior deterministic; accepted FIFO-operation terms align occupancy with actions that actually occur; derived widths allow parameters to change safely; and the documented stale-data rule prevents a blocked read from being mistaken for a valid transfer.

Maintaining specifications beside the RTL also improves review quality. A code change to state timing, flag behavior, or parameter limits becomes visible as a required specification and test-plan update rather than an isolated implementation detail.

Verification Follow-Up

Each cleanup item suggests a corresponding check. The sequence detector should assert that `detected` is known and one cycle wide. The traffic-light controller should assert positive durations, legal states, correct counter ranges, and mutual exclusion of greens. The FIFO should assert occupancy bounds, flag equivalence, stable pointers on blocked requests, unchanged `dout` on empty reads, and correct count behavior for every accepted read/write combination.

Key Learning

Writing or completing the specification after an initial RTL implementation is not merely administrative work. It exposes missing reset values, incomplete assignments, ambiguous output behavior, unsafe parameter assumptions, and corner cases that code review alone may overlook. This specification-driven cleanup creates a feedback loop in which the document improves the RTL and the RTL, in turn, makes the document more precise.

Conclusion

Day 8 consolidated the three design specifications and used them as active engineering tools for RTL cleanup. The comparison identified incomplete output defaults, simulation-only logic in synthesizable files, incorrectly qualified FIFO operations, undocumented empty-read behavior, and hardcoded counter sizing. The proposed corrections improve determinism, synthesis portability, parameterization, and verification readiness. Most importantly, the exercise showed that specification and implementation should evolve together under source control: a precise document reveals design gaps early and provides the observable requirements needed to prove that each cleanup change is correct.

Day 9: Feature Refinement, Requirements & Traceability

Objective

The objective of Day 9 was to refine the synchronous FIFO feature list into individually verifiable requirements, assign suitable test categories, define entry and exit criteria for sequence-detector verification, and demonstrate traceability from features through requirements, tests, and coverage.

Procedure

FIFO Feature Refinement

The initial Day 3 features were reviewed so that each final feature described one observable behavior. Broad read and write features were separated into accepted and blocked operations, flag assertion was separated from deassertion, and simultaneous-operation and pointer-boundary behavior were added explicitly.

Original feature	Refined feature	Rationale
Write operation	Valid and blocked writes	Distinguishes normal storage from overflow protection.
Read operation	Valid and blocked reads	Distinguishes normal removal from underflow protection.
Full-flag behavior	Full assertion and deassertion	Defines both boundary transitions explicitly.
Empty-flag behavior	Empty assertion and deassertion	Defines both boundary transitions explicitly.
Overflow protection	Write blocking when full	Merged with the blocked-write requirement.
New	Simultaneous read and write	Covers concurrent pointer and count logic.
New	Pointer wraparound	Covers memory-address boundaries independently.

Final FIFO Requirements and Test Categories

All numeric entries in the first column use the prefix `FIFO_REQ_`.

Req-ID	Feature requirement	Category	Verification purpose
001	Valid write when `wr_en=1` and not full	Functional	Confirms fundamental storage behavior.
002	Block write while full	Negative / corner	Prevents overflow and corruption.
003	Valid read when `rd_en=1` and not empty	Functional	Confirms fundamental removal behavior.
004	Block read while empty	Negative / corner	Prevents underflow and invalid transfer.
005	Assert `full` on reaching `DEPTH`	Functional / timing	Checks flow-control timing.
006	Deassert `full` after a valid read	Functional / timing	Confirms immediate availability of new space.
007	Assert `empty` after draining the FIFO	Functional / timing	Checks the zero-occupancy boundary.
008	Deassert `empty` after a valid write	Functional / timing	Confirms immediate data availability.
009	Simultaneous valid read and write	Stress / concurrent	Checks unchanged occupancy and data integrity.
010	Read- and write-pointer wraparound	Boundary / corner	Checks memory addressing at `DEPTH-1`.
011	Preserve first-in-first-out data order	Data integrity	Confirms the defining FIFO property.
012	Reset clears logical state	Reset	Confirms pointers and occupancy return to the empty condition.

Sequence-Detector Entry Criteria

Verification may begin when the RTL compiles without syntax errors, lint and applicable structural checks are reviewed, the clock/reset testbench and waveform capture are operational, a basic reset plus nominal-sequence smoke test passes, and the verification plan has been reviewed. Because this is a single-clock block, a conventional CDC analysis is normally not applicable unless the detector is integrated with asynchronous external logic.

Sequence-Detector Exit Criteria

The project exit targets are:

- complete statement, branch, toggle, and FSM coverage for reachable and meaningful implementation behavior;
- complete functional coverage for all approved coverpoints and bins;
- all assertions passing and the full regression suite green;

- every directed corner case passing, including continuous ones, partial matches, reset interruption, and overlap;
- at least 1000 constrained-random iterations without an unexplained failure;
- all P0 and P1 defects fixed and verified, with any accepted lower priority issue documented; and
- the verification report reviewed and approved.

Coverage that is proven unreachable or irrelevant must be formally reviewed and waived rather than silently preventing closure.

Example Traceability Matrix

Feature	Req-ID	Test	Coverage item
Valid write	001	Write sequential data until full and compare later reads with a reference queue.	Valid-write bins crossed with occupancy increments.
Full assertion	005	Fill to capacity and check assertion after the last accepted write.	Full-rise bin crossed with count at `DEPTH`.
Simultaneous read/write	009	Apply both requests across fill levels and compare data plus count.	Both-on control cross and occupancy bins showing no change when both operations are accepted.

Observations and Results

The refined list expanded a superficially simple FIFO into twelve measurable requirements. This prevented overflow protection from being hidden inside a generic write test and made flag deassertion, concurrency, wraparound, ordering, and reset independently traceable. The example matrix demonstrated how a requirement is considered covered only when it has both a checking mechanism and a measurable coverage item.

Conclusion

Day 9 transformed broad design features into precise requirements and linked them to test intent and coverage. Clear entry and exit criteria define when verification can start and what objective evidence is required for closure. The traceability approach reduces omissions and makes verification status auditable from specification through final results.

Day 10: Checker Types, Stimulus & Priority

Objective

The objective of Day 10 was to choose an appropriate checker, stimulus style, and priority for each FIFO requirement; define safety assertions for the traffic-light controller; and explain the roles of a FIFO scoreboard and mixed directed/constrained-random stimulus.

Procedure

FIFO Verification Strategy Table

Requirement numbers below use the prefix `FIFO_REQ_`.

Req-ID	Feature	Checker	Stimulus	Pri.	Reason
001	Valid write	Scoreboard	Random	P0	Fundamental storage
002	Blocked full write	Assertion and directed check	Directed	P0	Prevents corruption
003	Valid read	Scoreboard	Random	P0	Fundamental removal
004	Blocked empty read	Assertion and directed check	Directed	P0	Prevents underflow
005	Full assertion	Expected-value checker	Directed	P0	Flow-control boundary
006	Full deassertion	Expected-value checker	Directed	P0	Restores write availability
007	Empty assertion	Expected-value checker	Directed	P0	Zero-entry boundary
008	Empty deassertion	Expected-value checker	Directed	P0	Restores read availability
009	Simultaneous operations	Scoreboard	Constrained random	P0	Complex concurrency
010	Pointer wrap	Expected-value checker	Directed	P1	Boundary addressing
011	Data ordering	Scoreboard	Random	P0	Defining FIFO property
012	Reset behavior	Expected-value checker	Directed	P0	Fundamental recovery

Scoreboards are used where the expected data stream must be modeled. Assertions are suited to properties that must always hold, such as pointer stability during a blocked request. Expected-value checkers are effective for deterministic flag, reset, and boundary tests.

Traffic-Light Controller Assertions

1. **Mutual exclusion of green:** Main-road green and side-road green must never be asserted simultaneously.
2. **Legal one-hot light encoding:** Exactly one of red, yellow, and green must be active for each road. During the main-road yellow phase, for example, main yellow is active while main red and main green are inactive.
3. **Legal FSM state:** The registered state must always be one of `MAIN_GREEN`, `MAIN_YELLOW`, `SIDE_GREEN`, or `SIDE_YELLOW` after reset is released.

The second assertion corrects the ambiguous idea that a road must always be either red or green; yellow is itself a valid exclusive indication.

FIFO Scoreboard

The scoreboard maintains a software queue representing the FIFO's expected contents. An accepted write pushes the monitored input word. An accepted read pops the oldest expected word and compares it with the DUT's registered `dout`. Queue length predicts occupancy, `full`, and `empty`. A mismatch can therefore expose storage corruption, ordering errors, pointer mistakes, incorrect request blocking, or flag timing errors.

The model must update from accepted operations, not raw enables. At full, `wr_en` alone does not push the reference queue; at empty, `rd_en` alone does not pop it.

Why Directed and Random Stimulus Are Both Required

Directed tests guarantee that known high-risk scenarios occur and provide short, deterministic failures that are easy to debug. They are ideal for reset, full and empty transitions, blocked requests, exact pointer wrap, and parameter boundaries. Constrained-random stimulus explores combinations and timing relationships that were not manually anticipated, including burst patterns and back-to-back concurrency across many occupancy levels. Together they cover known risks and search for unknown interactions.

Observations and Results

The strategy assigns P0 to behaviors whose failure makes the FIFO unsafe or unusable. Pointer wrap is marked P1 because it is a boundary case, although it must still pass before a reusable FIFO is signed off. Pairing each feature with the most natural checker avoids forcing temporal safety properties into a scoreboard or using assertions as a substitute for a data-order reference model.

Conclusion

Day 10 established a risk-based verification strategy. Scoreboards protect data integrity, assertions enforce invariant behavior, expected-value checkers cover deterministic results, and complementary stimulus styles balance debugability with exploration. Correct traffic-light assertions also showed why properties must reflect every legal output state, including yellow.

Day 11: Coverage Theory & Coverage Planning

Objective

The objective of Day 11 was to define meaningful functional coverage for the sequence detector and FIFO, relate coverage items to requirements, and explain the complementary roles of code coverage and specification-based functional coverage.

Procedure

Sequence-Detector Coverage Model

State coverage: Record entry into `IDLE`, `S1`, `S10`, and `S101`.

Transition coverage: Cover `IDLE`→`IDLE`, `IDLE`→`S1`, `S1`→`S1`, `S1`→`S10`, `S10`→`IDLE`, `S10`→`S101`, `S101`→`S10`, and the overlap-preserving detection transition `S101`→`S1` when `din=1`.

Output coverage: Cover at least one detection and cross detection with `S101` plus `din=1`. Also cover a low output in every non-detection condition. This matches the four-state Mealy implementation.

Pattern coverage: Cover `1011`, `10110`, `1011011`, continuous ones, and continuous zeros.

FIFO Occupancy Coverage

Bin	Values	Purpose
`bin_empty`	0	Exercises empty flag and underflow blocking.
`bin_one`	1	Covers the single-entry boundary.
`bin_mid`	1 to DEPTH−1	Exercises normal intermediate levels.
`bin_dep_minus_one`	DEPTH−1	Covers the last free entry.
`bin_full`	DEPTH	Exercises full flag and overflow blocking.

Overlapping bins such as `bin_one` and `bin_mid` are useful when the coverage language permits them because the named boundary retains its own coverage goal while also contributing to the general mid-range view.

Cross Coverage: `wr_en` by `rd_en`

Bin	wr_en	rd_en	Meaning
`both_off`	0	0	Idle stability
`write_only`	1	0	Storage, pointer advance, count increase
`read_only`	0	1	Output, pointer advance, count decrease
`both_on`	1	1	Concurrent operation and boundary arbitration

This control cross should also be crossed with occupancy class because `both_on` behaves differently at empty, middle, and full conditions.

FIFO Requirement-to-Coverage Table

Requirement numbers below use the prefix `FIFO_REQ_`.

Req-ID	Feature	Coverage intent
001	Valid write	Accepted-write bins crossed with occupancy
002	Blocked full write	Full plus write-request bin
003	Valid read	Accepted-read bins crossed with occupancy
004	Blocked empty read	Empty plus read-request bin
005	Full assertion	Count-to-DEPTH and full rising transition
006	Full deassertion	Full-to-not-full transition after read
007	Empty assertion	Count-to-zero and empty rising transition
008	Empty deassertion	Empty-to-not-empty transition after write
009	Simultaneous operations	Both-on crossed with empty, middle, and full occupancy
010	Pointer wrap	Separate write-pointer-wrap and read-pointer-wrap bins
011	Data order	Coverage records read/write traffic and data classes; the scoreboard, not coverage, proves ordering correctness.
012	Reset	Reset event crossed with nonzero occupancy and return to empty

Branch Coverage versus Functional Coverage

Branch coverage is implementation-based: it reports whether each conditional outcome in the RTL executed. Functional coverage is intent-based: it reports whether specification scenarios selected by the DV engineer occurred. A design can execute every branch without testing overlap, pointer wrap, or flag timing correctly; conversely, planned scenarios may all occur while an implementation error-recovery branch remains untouched. Coverage also does not prove that the output was correct—checkers and assertions provide that proof. Verification closure therefore requires complementary code coverage, functional coverage, and passing check results.

Observations and Results

The coverage plan emphasized boundaries and crosses rather than collecting only simple signal values. Correcting the sequence transition to `S101`→`S1` and treating detection as Mealy-style kept the coverage model consistent with the selected RTL. The FIFO plan similarly distinguished scenario measurement from scoreboard-based correctness checking.

Conclusion

Day 11 established coverage as a measurable view of verification intent rather than a substitute for checking. State, transition, pattern, occupancy, control, flag, and pointer-wrap coverage together reveal which meaningful situations have occurred. Code and functional coverage remain complementary and must be interpreted alongside assertions, scoreboards, and regression results.

Day 12: Knowledge-Base Completion & Self-Review

Objective

The objective of Day 12 was to audit traceability across all three designs, close missing coverage links, review the cumulative issue log, and complete the knowledge base before final reflection and presentation preparation.

Procedure

Traceability Review

Design	Count	Req-IDs	Coverage	Gap and resolution
Sequence detector	4	SEQ_REQ_001–004	State, transition, output, reset	Added low-output coverage for every non-detection condition.
Traffic light	4	TLC_REQ_001–004	State, transition, duration, output	Added bins for all four configured state durations.
FIFO	12	FIFO_REQ_001–012	Twelve linked coverage intents	Split broad pointer-wrap coverage into independent write- and read-wrap bins.

Traceability Gaps Closed

- Traffic-light duration coverage was added for each individual phase so state entry alone could not conceal an incorrect holding time.
- FIFO pointer coverage was separated into `wr_ptr_wrap` and `rd_ptr_wrap`, allowing each address path to close independently.
- Sequence-detector output coverage was expanded to confirm that `detected=0` in every non-detection combination as well as high for a valid final bit.

Issue-Log Review

The final issue-log review required every entry to contain a bug identifier, date or day, symptom, root cause, correction, verification method, and final status. Earlier issues involving counter terminal handling, stale outputs, FIFO occupancy, simultaneous operations, default assignments, and parameter sizing were checked for complete documentation. The assignment record refers to `issues/ISSUE_LOG.md`; that repository artifact should be verified alongside the report before submission.

Observations and Results

Traceability was complete only after each requirement had a test/checking path and a coverage measurement. The review showed that broad coverage items can hide gaps: entering a traffic state does not prove its full duration, and seeing a pointer reach its maximum does not prove both pointer wrap paths.

Conclusion

Day 12 completed the knowledge-base audit and refined the remaining coverage gaps. Independent duration, pointer-wrap, and non-detection bins made the traceability matrix more precise. The issue-log review also reinforced that a fix is not complete until its root cause, verification method, and final status are documented.

Day 13: Final Team Reflection

Objective

The objective of Day 13 was to reflect on the technical and process lessons of the assignment from the perspectives of RTL design, design verification, and team leadership, while recording unresolved questions for future study.

Procedure

RTL Designer Reflection

The two-block FSM structure—registered state plus combinational next-state logic—became intuitive and improved readability and debug. The most difficult task was correct FIFO occupancy handling during simultaneous operations. The case-based update using accepted read and write terms resolved this complexity. The central lesson was that a specification is an engineering and verification tool, not merely documentation. In a future project, the specification would be written before RTL to reduce rework. The remaining question concerned how to close coverage when a bin is impossible because of design constraints.

DV Engineer Reflection

The distinction between directed, random, and constrained-random stimulus became clear, as did the practical value of P0/P1/P2 prioritization. Coverage-model design was the hardest task because useful bins require deep understanding of design intent. The most important lesson was to verify against the specification rather than treating the RTL as the source of truth. Earlier cross-review between DV and RTL engineers would be adopted next time. The remaining question was how real projects decide that verification evidence is sufficient for tape-out.

Team-Lead Reflection

The complete path from RTL design through planning, coverage, and traceability became coherent. Scope management was difficult because the FIFO expanded from five broad features to twelve precise requirements. The strongest lesson was that documentation and traceability are foundations of verification closure. More time would be allocated to specification development in the next project. The open question was how formal verification should be integrated with simulation and which properties benefit most from formal methods.

Observations and Results

Responses to the Open Questions

Impossible coverage bins should be analyzed, justified, and excluded or waived through a reviewed coverage-closure process; the goal is not to fabricate a hit but to prove why the scenario is unreachable. Tape-out readiness is a joint engineering and management decision based on approved requirements, passing regressions, coverage closure with reviewed exclusions, assertion status, remaining defect risk, and sign-off from responsible design, DV, and project owners. Formal verification is particularly valuable for invariants and bounded control properties such as traffic-light mutual exclusion, legal FSM states, FIFO count bounds, blocked-pointer stability, and flag equivalence. Simulation remains essential for long data scenarios, software interaction, performance, and system-level behavior.

Conclusion

Day 13 showed that the assignment's most important progress was not only the completion of three RTL blocks but the development of shared design and verification reasoning. The reflections consistently favored earlier specification work, earlier cross-team review, explicit risk prioritization, and evidence-based closure. The remaining questions naturally lead toward formal verification, sign-off methodology, and professional coverage-waiver practice.

Day 14: Presentation Preparation & Final Wrap-Up

Objective

The objective of the final day was to organize a concise presentation of the design and verification work, review the declared deliverables, and summarize the full 14-day learning journey.

Procedure

Presentation Walkthrough

Time	Section	Content
5 min	Design walkthrough	Three designs, key trade-offs, and microarchitecture.
5 min	Specification walkthrough	Document structure, interfaces, and corner cases.
5 min	Verification strategy	Refined features, requirements, checkers, stimulus, priorities, and coverage.
3 min	Traceability and metrics	Traceability matrix, coverage closure, and issue-log review.
2 min	Lessons and questions	Main takeaways and future directions.

Final Deliverables Checklist

The following table records the assignment's declared deliverables. Because this LaTeX workspace contains the report rather than the complete external repository, each referenced path should be checked in the submission package before final handoff.

Deliverable	Declared location	Status
RTL code for three designs	`rtl/`	Complete
RTL specifications	`docs/RTL_Specifications.docx`	Complete
Knowledge base	`docs/Knowledge_Base.docx`	Complete
Issue log	`issues/ISSUE_LOG.md`	Complete
Smoke-test logs	`docs/experiments/`	Complete
State diagrams	`docs/experiments/`	Complete
Coverage plan	`docs/coverage_plan.md`	Complete
Traceability matrix	`docs/traceability_matrix.md`	Complete
Presentation	`docs/presentation.pptx`	Ready

Knowledge-Base Final Entry

The 14-day assignment covered the complete introductory RTL design and verification lifecycle. It began with synthesis, coding constructs, FSM design, and testbench architecture; progressed through implementation and initial simulation of a sequence detector, timed traffic-light controller, and synchronous FIFO; and concluded with specification writing, corner-case review, requirements, verification planning, coverage, traceability, and cleanup.

The FIFO became the most verification-intensive block because concurrent operations, occupancy boundaries, pointer wrap, flag timing, ordering, reset, and blocked requests all interact. Refining its feature list from five broad items to twelve requirements demonstrated why verification scope grows as behavior becomes more precise. The traceability matrix connected each feature to a requirement, test/checker, and coverage item, providing objective evidence for sign-off.

Observations and Results

The final presentation structure allocates equal emphasis to implementation, specification, and verification rather than treating verification as an afterthought. The checklist also makes final submission risk visible: a report may describe an artifact, but the actual file, revision, and path must still be confirmed in the repository.

Conclusion

The completed assignment demonstrated that reliable digital design requires RTL and verification mindsets working together from the beginning. Clear specifications provide the source of truth; RTL realizes that intent; testbenches, assertions, scoreboards, and coverage provide evidence; and traceability connects the evidence back to every requirement. Across fourteen days, the work advanced from basic synthesizable coding to disciplined verification planning and closure. The lasting lesson is that verification is an integral design activity whose quality depends on early planning, precise corner-case behavior, measurable coverage, independent checking, and complete documentation.